

机器学习因果推断之机器学习篇

从零开始精通机器学习应用

3.1 随机森林原理与经济学应用

吕斌

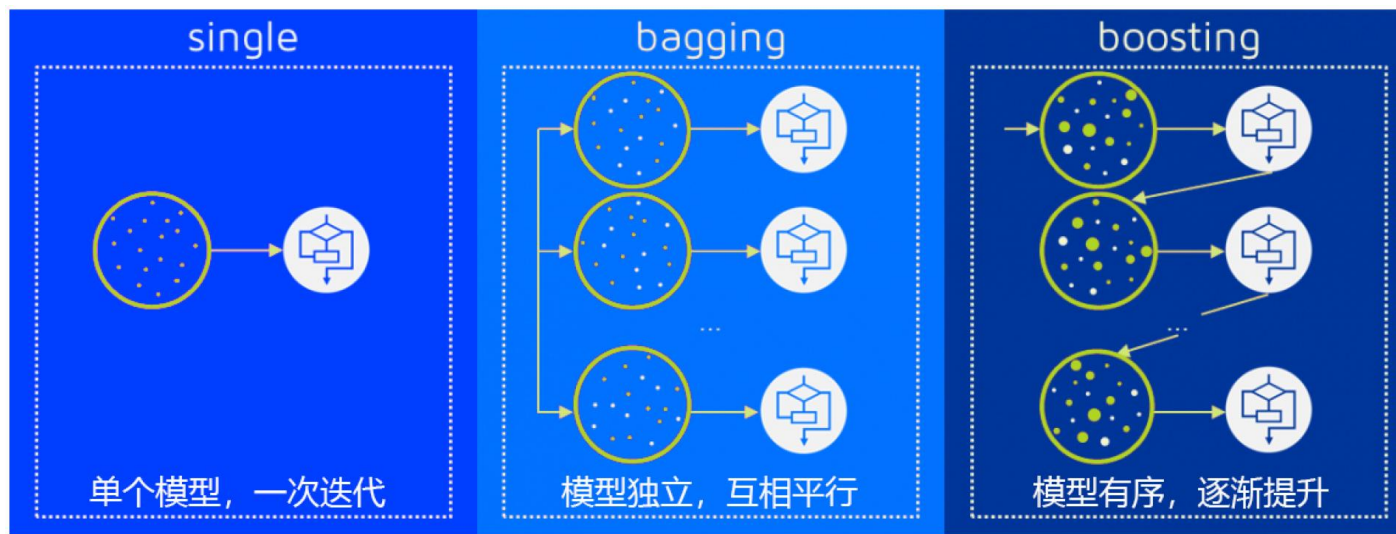
上海外国语大学国际金融贸易学院

集成学习

- 集成学习(Ensemble Learning) 的方法，也称为组合学习。集成学习就是将预测效果一般的弱学习器，以某种方式将它们组合起来，构成一个预测效果优良的强学习器(Strong Learner)。
- 集成学习的基本思路类似于“三个臭皮匠，顶个诸葛亮” (Wisdom of Crowds) 。
- 用于集成学习的弱学习器，也称为基学习器(Base Learner)，决策树是最常见的基学习器。
- 现在的各种算法竞赛中，随机森林，梯度提升树 (GBDT)， Xgboost等集成学习算法的身影也随处可见，可见其效果之好，应用之广。
- 集成算法的目标：集成算法会考虑多个评估器的建模结果，汇总之后得到一个综合的结果，以此来获取比单个模型更好的回归或分类表现。

集成学习

- 多个模型集成的模型叫做集成评估器（Ensemble estimator），组成集成评估器的每个模型都叫做基评估器（Base estimator）。通常来说，有三类集成算法：装袋法（Bagging）、提升法（Boosting）、堆叠法（Stack）。
- 装袋法的核心思想是构建多个相互独立的评估器，然后对其预测进行平均或多数表决原则来决定集成评估器的结果。装袋法的代表模型是随机森林。从方差-偏差的角度看，可以有效减小方差。
- 提升法中，基评估器是相关的，是按顺序一一构建的。其核心思想是结合弱评估器的力量一次次对难以评估的样本进行预测，从而使得难评估样本的权重增大，从而构成一个强评估器。提升法的代表模型有Adaboost和梯度提升树。



Sklearn中的集成算法

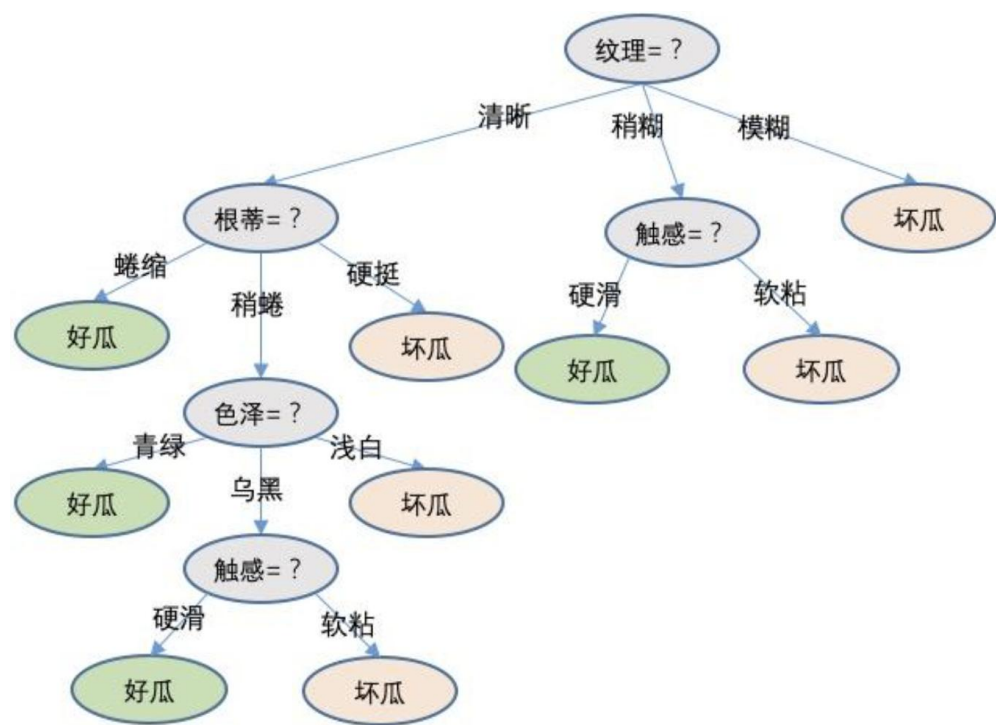
- 集成算法中，有一半以上都是树的集成模型，可见决策树在集成中必定是有很好的效果。本节课将会以随机森林为例，讲解集成算法。

类	类的功能
<code>ensemble.AdaBoostClassifier</code>	AdaBoost分类
<code>ensemble.AdaBoostRegressor</code>	Adaboost回归
<code>ensemble.BaggingClassifier</code>	装袋分类器
<code>ensemble.BaggingRegressor</code>	装袋回归器
<code>ensemble.ExtraTreesClassifier</code>	Extra-trees分类（超树，极端随机树）
<code>ensemble.ExtraTreesRegressor</code>	Extra-trees回归
<code>ensemble.GradientBoostingClassifier</code>	梯度提升分类
<code>ensemble.GradientBoostingRegressor</code>	梯度提升回归
<code>ensemble.IsolationForest</code>	隔离森林
<code>ensemble.RandomForestClassifier</code>	随机森林分类
<code>ensemble.RandomForestRegressor</code>	随机森林回归
<code>ensemble.RandomTreesEmbedding</code>	完全随机树的集成
<code>ensemble.VotingClassifier</code>	用于不合适估算器的软投票/多数规则分类器

回顾决策树

- 决策树是一种原理简单，应用广泛的模型，它可以同时被用于分类和回归问题。
- 决策树的主要功能是从一个有特征和标签的表格中，通过对特定特征进行提问，总结出一系列决策规则，并用树状图来呈现这些决策规则。

	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否

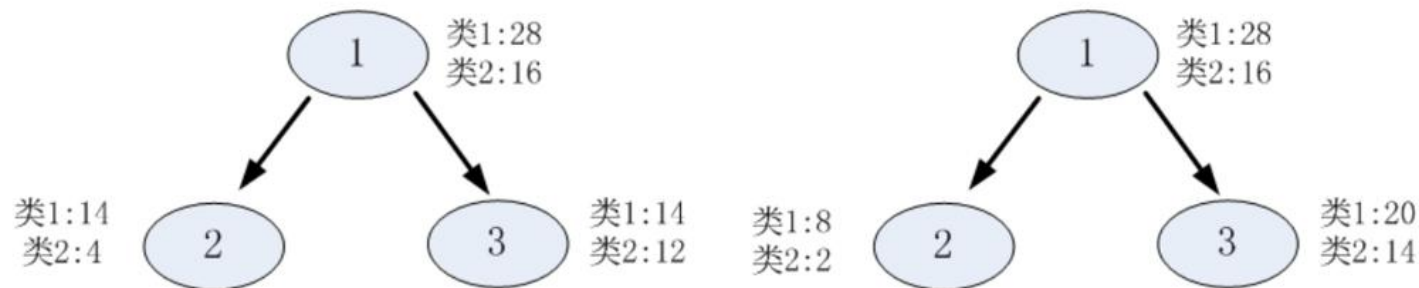


回顾决策树

- 决策树的核心问题有两个：
- 一、如何找出正确的特征来进行提问，即如何分枝？
- 二、树生长到什么时候应该停下？
- 对于第一个问题，定义了用来衡量分枝质量的指标不纯度。分类树的不纯度用基尼系数或信息熵来衡量，回归树的不纯度用MSE均方误差来衡量。每次分枝时，决策树对所有的特征进行不纯度计算，选取不纯度最低的特征进行分枝，分枝后，又再对被分枝的不同取值下，计算每个特征的不纯度，继续选取不纯度最低的特征进行分枝。



回顾决策树



$$Entropy(t) = - \sum_{i=0}^{c-1} p(i|t) \log_2 p(i|t)$$

$$Gini(t) = 1 - \sum_{i=0}^{c-1} p(i|t)^2$$

$$Info_1 = Entropy - \sum_{i=1}^n Entropy_i =$$

$$\frac{16}{28+16} \cdot \log\left(\frac{16}{28+16}\right) + \frac{28}{28+16} \cdot \log\left(\frac{28}{28+16}\right) \Rightarrow Entropy$$

$$-\left(\frac{14}{14+4} \cdot \log\left(\frac{14}{14+4}\right) + \frac{4}{14+4} \cdot \log\left(\frac{4}{14+4}\right)\right) \Rightarrow Entropy1$$

$$-\left(\frac{14}{14+12} \cdot \log\left(\frac{14}{14+12}\right) + \frac{12}{14+12} \cdot \log\left(\frac{12}{14+12}\right)\right) \Rightarrow Entropy2$$

$$= 0.81$$

$$Info_2 = Entropy - \sum_{i=1}^n Entropy_i =$$

$$\frac{16}{28+16} \cdot \log\left(\frac{16}{28+16}\right) + \frac{28}{28+16} \cdot \log\left(\frac{28}{28+16}\right) \Rightarrow Entropy$$

$$-\left(\frac{8}{8+2} \cdot \log\left(\frac{8}{8+2}\right) + \frac{2}{8+2} \cdot \log\left(\frac{2}{8+2}\right)\right) \Rightarrow Entropy1$$

$$-\left(\frac{20}{20+14} \cdot \log\left(\frac{20}{20+14}\right) + \frac{14}{20+14} \cdot \log\left(\frac{14}{20+14}\right)\right) \Rightarrow Entropy2$$

$$= 0.75$$

- $Info1 > Info2$, 所以属性1与属性2相比是更优的分裂属性, 选择属性1作为分裂属性。

回顾决策树

- 决策树的核心问题有两个：
- 一、如何找出正确的特征来进行提问，即如何分枝？
- 二、树生长到什么时候应该停下？
- 对于第二个问题，每分枝一层，树整体的不纯度会越来越小，决策树追求的是最小不纯度。因此，决策树会一直分枝，直到没有更多的特征可用，或整体的不纯度指标已经最优，决策树就会停止生长。
- 决策树非常容易过拟合，很容易在训练集上表现优秀，却在测试集上表现糟糕。为了防止决策树的过拟合，要对决策树进行剪枝，sklearn中提供了大量的剪枝参数。

```
class sklearn.tree.DecisionTreeClassifier (criterion='gini', splitter='best', max_depth=None,  
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=None,  
random_state=None, max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None,  
class_weight=None, presort=False)
```


随机森林分类器

RandomForestClassifier

随机森林是非常具有代表性的bagging集成算法，它的所有基评估器都是决策树，分类树组成的森林就叫做随机森林分类器，回归树所集成的森林就叫做随机森林回归器。这些参数在随机森林中的含义，和在决策树中一模一样。

```
class sklearn.ensemble.RandomForestClassifier (n_estimators='10', criterion='gini', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='auto', max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None, bootstrap=True, oob_score=False, n_jobs=None, random_state=None, verbose=0, warm_start=False, class_weight=None)
```

参数	含义
criterion	不纯度的衡量指标，有基尼系数和信息熵两种选择
max_depth	树的最大深度，超过最大深度的树枝都会被剪掉
min_samples_leaf	一个节点在分枝后的每个子节点都必须包含至少min_samples_leaf个训练样本，否则分枝就不会发生
min_samples_split	一个节点必须要包含至少min_samples_split个训练样本，这个节点才允许被分枝，否则分枝就不会发生
max_features	max_features限制分枝时考虑的特征个数，超过限制个数的特征都会被舍弃，默认值为总特征个数开平方取整
min_impurity_decrease	限制信息增益的大小，信息增益小于设定数值的分枝不会发生

重要参数n_estimators

- 这是森林中树木的数量，即基评估器的数量。这个参数对随机森林模型的精确性影响是单调的，n_estimators越大，模型的效果往往越好。但是相应的，任何模型都有决策边界，n_estimators达到一定的程度之后，随机森林的精确性往往不在上升或开始波动。
- n_estimators越大，需要的计算量和内存也越大，训练的时间也会越来越长。对于这个参数，希望在训练难度和模型效果之间取得平衡。
- n_estimators的默认值在sklearn0.22版本以前是10，在0.22版本以后，默认值被修正为100。
- 修正显示出了使用者的调参倾向：要更大的n_estimators。

建立随机森林

- 树模型的优点是简单易懂，可视化之后的树人人都能够看懂，可惜随机森林是无法被可视化的。所以为了更加直观地展示随机森林的效果，进行一个随机森林和单个决策树效益的对比。使用红酒数据集。

- 1、导入本次案例中需要的包。

```
%matplotlib inline
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_wine
```

- 2、导入本次案例中需要的数据集

```
wine = load_wine()

wine.data
wine.target
```

建立随机森林

- 3、Sklearn建立随机森林模型。

```
from sklearn.model_selection import train_test_split

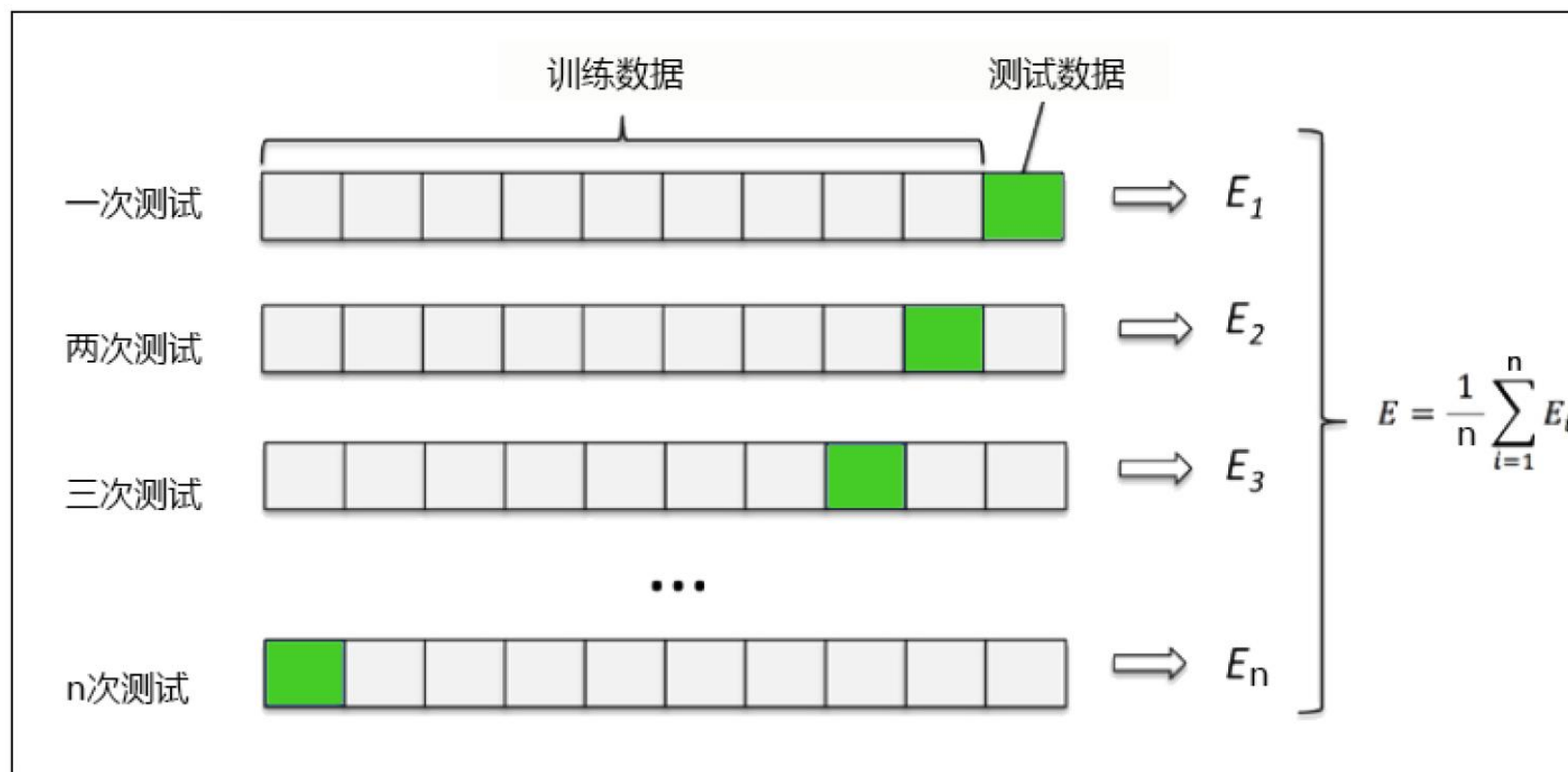
Xtrain, Xtest, Ytrain, Ytest = train_test_split(wine.data, wine.target, test_size=0.3)

clf = DecisionTreeClassifier(random_state=0)
rfc = RandomForestClassifier(random_state=0)
clf = clf.fit(Xtrain, Ytrain)
rfc = rfc.fit(Xtrain, Ytrain)
score_c = clf.score(Xtest, Ytest)
score_r = rfc.score(Xtest, Ytest)

print("Single Tree:{}".format(score_c)
      , "Random Forest:{}".format(score_r)
      )
```

建立随机森林

- 4、交叉验证评估效果。



建立随机森林

- 4、交叉验证评估效果。

#交叉验证：是数据集划分为n分，依次取每一份做测试集，每n-1份做训练集，多次训练模型以观测模型稳定性的方法

```
from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt

rfc = RandomForestClassifier(n_estimators=25)
rfc_s = cross_val_score(rfc,wine.data,wine.target,cv=10)

clf = DecisionTreeClassifier()
clf_s = cross_val_score(clf,wine.data,wine.target,cv=10)

plt.plot(range(1,11),rfc_s,label = "RandomForest")
plt.plot(range(1,11),clf_s,label = "Decision Tree")
plt.legend()
plt.show()
```

建立随机森林

- 5、多次交叉验证平均结果绘图。

```
rfc_l = []
clf_l = []

for i in range(10):
    rfc = RandomForestClassifier(n_estimators=25)
    rfc_s = cross_val_score(rfc,wine.data,wine.target,cv=10).mean()
    rfc_l.append(rfc_s)
    clf = DecisionTreeClassifier()
    clf_s = cross_val_score(clf,wine.data,wine.target,cv=10).mean()
    clf_l.append(clf_s)

plt.plot(range(1,11),rfc_l,label = "Random Forest")
plt.plot(range(1,11),clf_l,label = "Decision Tree")
plt.legend()
plt.show()
```

#是否有注意到，单个决策树的波动轨迹和随机森林一致？

#再次验证了我们之前提到的，单个决策树的准确率越高，随机森林的准确率也会越高

建立随机森林

- 6、n_estimators的学习曲线。

```
##### 【TIME WARNING: 2mins 30 seconds】 #####
```

```
superpa = []  
for i in range(200):  
    rfc = RandomForestClassifier(n_estimators=i+1,n_jobs=-1)  
    rfc_s = cross_val_score(rfc,wine.data,wine.target,cv=10).mean()  
    superpa.append(rfc_s)  
print(max(superpa),superpa.index(max(superpa)))  
plt.figure(figsize=[20,5])  
plt.plot(range(1,201),superpa)  
plt.show()
```

为什么群体智慧一定有效？

- 随机森林用了什么方法，来保证集成的效果一定好于单个分类器？
- 随机森林的本质是一种装袋集成算法（bagging），装袋集成算法是对基评估器的预测结果进行平均或用多数表决原则来决定集成评估器的结果。在刚才的红酒例子中，我们建立了25棵树，对任何一个样本而言，平均或多数表决原则下，当且仅当有13棵以上的树判断错误的时候，随机森林才会判断错误。单独一棵决策树对红酒数据集的分类准确率在0.8上下浮动，假设一棵树判断错误的可能性为0.2(ϵ)，那13棵树以上都判断错误的可能性是：

$$e_{\text{random_forest}} = \sum_{i=13}^{25} C_{25}^i \epsilon^i (1 - \epsilon)^{25-i} = 0.000369$$

- 其中， i 是判断错误的次数，也是判错的树的数量， ϵ 是一棵树判断错误的概率， $(1-\epsilon)$ 是判断正确的概率，共判对 $(25-i)$ 次。采用组合，是因为25棵树中，有任意 i 棵都判断错误。可见，判断错误的几率非常小，这让随机森林在红酒数据集上的表现远远好于单棵决策树。

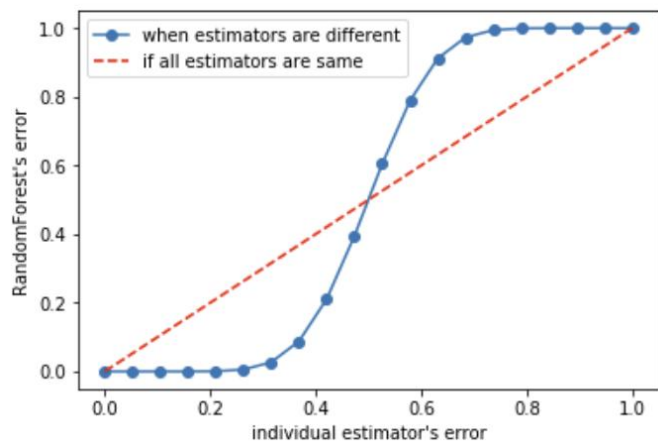
```
import numpy as np
from scipy.special import comb

np.array([comb(25,i)*(0.2**i)*((1-0.2)**(25-i)) for i in range(13,26)]).sum()
```


群体智慧发挥作用的前提条件是什么？

- 在使用袋装法时要求基评估器要尽量独立。其实，袋装法还有另一个必要条件：基分类器的判断准确率至少要超过随机分类器，即时说，基分类器的判断准确率至少要超过50%。之前我们已经展示过随机森林的准确率公式，基于这个公式，画出了基分类器的误差率 ϵ 和随机森林的误差率之间的图像。
- 可以从图像上看出，当基分类器的误差率小于0.5，即准确率大于0.5时，集成的效果是比基分类器要好的。相反，当基分类器的误差率大于0.5，袋装的集成算法就失效了。所以在使用随机森林之前，一定要检查，用来组成随机森林的分类树们是否都有至少50%的预测正确率。

$$e_{random_forest} = \sum_{i=13}^{25} C_{25}^i \epsilon^i (1 - \epsilon)^{25-i} = 0.000369$$



```
import numpy as np

x = np.linspace(0,1,20)

y = []
for epsilon in np.linspace(0,1,20):
    E = np.array([comb(25,i)*(epsilon**i)*((1-epsilon)**(25-i))
                  for i in range(13,26)]).sum()

    y.append(E)

plt.plot(x,y,"o-",label="when estimators are different")
plt.plot(x,x,"--",color="red",label="if all estimators are same")
plt.xlabel("individual estimator's error")
plt.ylabel("RandomForest's error")
plt.legend()
plt.show()
```


群体智慧发挥作用的前提条件是什么？

- 森林中单棵树的分类强度（Strength）：每颗树的分类强度越大，则随机森林的分类性能越好。
- 森林中树之间的相关度（Correlation）：树之间的相关度越大，则随机森林的分类性能越差。
- 减小特征选择次数 m ，树的相关性和分类能力也会相应的降低；增大 m ，两者也会随之增大。所以关键问题是如何选择最优的 m 。

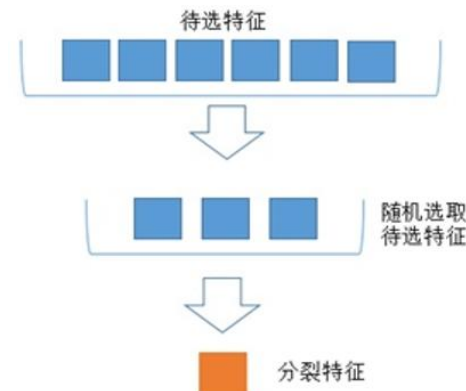
随机性来源于哪里？——随机森林双随机之随机特征

- Sklearn中的分类树DecisionTreeClassifier自带随机性，所以随机森林中的树天生就都是不一样的。在上节课讲解分类树时曾提到，决策树从最重要的特征中随机选择一个特征来进行分枝，因此每次生成的决策树都不一样，这个功能由参数random_state控制。
- 随机森林中其实也有random_state，用法和分类树中相似，只不过在分类树中，一个random_state只控制生成一棵树，而随机森林中的random_state控制的是生成森林的模式，而非让一个森林中只有一棵树。

```
rfc = RandomForestClassifier(n_estimators=20, random_state=2)
rfc = rfc.fit(Xtrain, Ytrain)
```

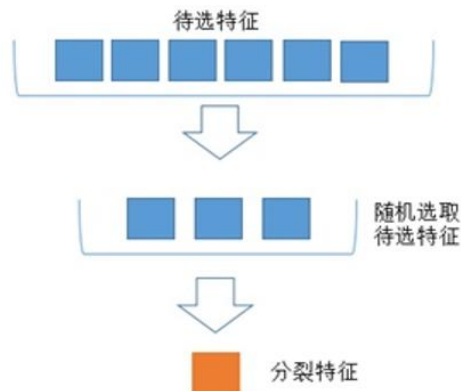
```
#随机森林的重要属性之一: estimators, 查看森林中树的状况
rfc.estimators_[0].random_state
```

```
for i in range(len(rfc.estimators_)):
    print(rfc.estimators_[i].random_state)
```



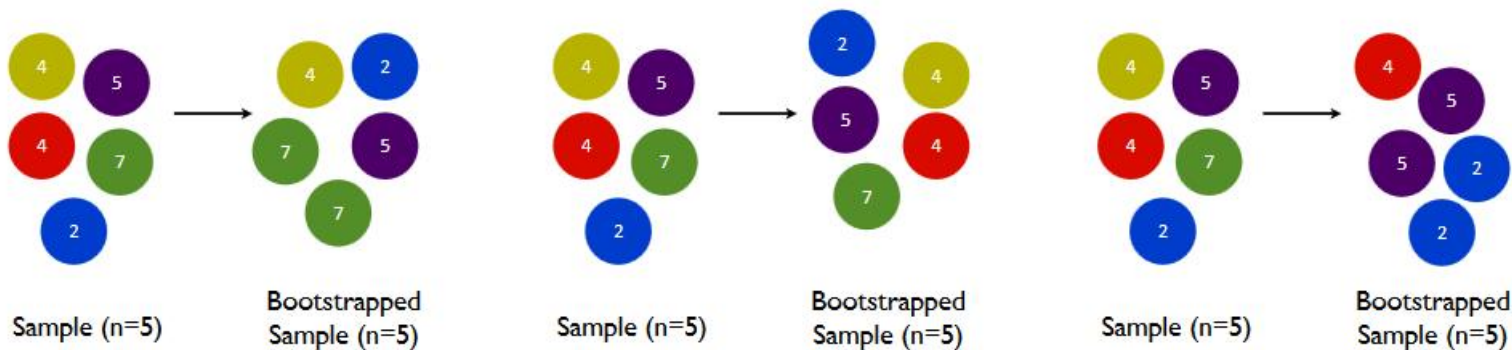
随机性来源于哪里？——随机森林双随机之随机特征

- 可以观察到，当random_state固定时，随机森林中生成是一组固定的树，但每棵树依然是不一致的，这是用“随机挑选特征进行分枝”的方法得到的随机性。并且我们可以证明，当这种随机性越大的时候，袋装法的效果一般会越来越好。用袋装法集成时，基分类器应当是相互独立的，是不相同的。
- 但这种做法的局限性是很强的，当我们需要成千上万棵树的时候，数据不一定能够提供成千上万的特征来让我们构筑尽量多尽量不同的树。因此，除了random_state之外，还需要其他的随机性。



随机性来源于哪里？——随机森林双随机之随机数据

- 要让基分类器尽量都不一样，一种很容易理解的方法是使用不同的训练集来进行训练，而袋装法正是通过有放回的随机抽样技术来形成不同的训练数据，Bootstrap就是用来控制抽样技术的参数。
- 在一个含有 n 个样本的原始训练集中进行随机采样，每次采样一个样本，并在抽取下一个样本之前将该样本放回原始训练集，也就是说下次采样时这个样本依然可能被采集到，这样采集 n 次，最终得到一个和原始训练集一样大的， n 个样本组成的自助集。由于是随机采样，这样每次的自助集和原始数据集不同，和其他的采样集也是不同的。这样就可以自由创造取之不尽用之不竭，并且互不相同的自助集，用这些自助集来训练基分类器，基分类器自然也就各不相同了。



随机性来源于哪里？——随机森林双随机之随机数据

- Bootstrap参数默认True，代表采用这种有放回的随机抽样技术。通常参数不会被设置为False。
- 然而有放回抽样也会有问题。由于是有放回，一些样本可能在同一个自助集中出现多次，而其他一些却可能被忽略，一般来说，自助集大约平均会包含63%的原始数据。因为每一个样本被抽到某个自助集中的概率为： $1 - (1 - \frac{1}{n})^n$
- 当n足够大时，这个概率收敛于 $1 - (1/e)$ ，约等于0.632。因此，会有约37%的训练数据被浪费掉，没有参与建模，这些数据被称为袋外数据(out of bag data, 简写为oob)。
- 除了最开始就划分好的测试集之外，这些数据也可以被用来作为集成算法的测试集。也就是说，在使用随机森林时，可以不划分测试集和训练集，只需要用袋外数据来测试模型即可。当然，这也不是绝对的，当n不够大的时候，很可能就没有数据掉落在袋外，自然也就无法使用oob数据来测试模型了。

#无需划分训练集和测试集

```
rfc = RandomForestClassifier(n_estimators=25, oob_score=True)
rfc = rfc.fit(wine.data, wine.target)
```

#重要属性oob_score_
rfc.oob_score_

重要属性和接口

- 至此，我们已经讲完了所有随机森林中的重要参数，为大家复习了一下决策树的参数，并通过`n_estimators`、`random_state`、`bootstrap`、`oob_score`四个参数帮助大家了解了袋装法的基本流程和重要概念。同时还介绍了`.estimators_`和`.oob_score_`两个重要属性。除了这两个属性之外，作为树模型的集成算法，随机森林自然也有`.feature_importances_`特征重要性属性。
- 随机森林的接口与决策树完全一致，因此依然有四个常用接口：`apply`、`fit`、`predict`和`score`。除此之外，还需要注意随机森林的`predict_proba`接口，这个接口返回每个测试样本对应的被分到每一类标签的概率，标签有几个分类就返回几个概率。如果是二分类问题，则`predict_proba`返回的数值大于0.5的，被分为1，小于0.5的，被分为0。

重要属性和接口

- **注意：**传统的随机森林是利用袋装法中的规则，平均或少数服从多数来决定集成的结果，而sklearn中的随机森林是平均每个样本对应的predict_proba返回的概率，得到一个平均概率，从而决定测试样本的分类。
- 重要属性和接口/方法/函数的算法实现：

重要属性和接口

```
rfc = RandomForestClassifier(n_estimators=25)
rfc = rfc.fit(Xtrain, Ytrain)
rfc.score(Xtest, Ytest)

rfc.feature_importances_
rfc.apply(Xtest)
rfc.predict(Xtest)
rfc.predict_proba(Xtest)
```

随机森林回归器

- 所有的参数，属性与接口，全部和随机森林分类器一致。仅有的不同就是回归树与分类树的不同，不纯度的指标，参数Criterion不一致。

```
class sklearn.ensemble.RandomForestRegressor (n_estimators='warn', criterion='mse', max_depth=None,  
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='auto',  
max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None, bootstrap=True, oob_score=False,  
n_jobs=None, random_state=None, verbose=0, warm_start=False)
```

随机森林回归器

```
class sklearn.ensemble.RandomForestRegressor (n_estimators='warn', criterion='mse', max_depth=None,
min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='auto',
max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None, bootstrap=True, oob_score=False,
n_jobs=None, random_state=None, verbose=0, warm_start=False)
```

- 回归树衡量分枝质量的指标，支持的标准有三种：
- 1、输入 “mse”使用均方误差mean squared error(MSE)，父节点和叶子节点之间的均方误差的差额将被用来作为特征选择的标准，这种方法通过使用叶子节点的均值来最小化L2损失。

$$MSE = \frac{1}{N} \sum_{i=1}^N (f_i - y_i)^2$$

- 2、输入 “friedman_mse”使用费尔德曼均方误差，这种指标使用弗里德曼针对潜在分枝中的问题改进后的均方误差。
- 3、输入 “mae”使用绝对平均误差MAE (mean absolute error) ，这种指标使用叶节点的中值来最小化L1损失。

随机森林回归器

$$MSE = \frac{1}{N} \sum_{i=1}^N (f_i - y_i)^2$$

- 其中N是样本数量，i是每一个数据样本， f_i 是模型回归出的数值， y_i 是样本点i实际的数值标签。所以MSE的本质是样本真实数据与回归结果的差异。
- 在回归树中，MSE不只是我们的分枝质量衡量指标，也是最常用的衡量随机森林回归质量的指标，当在使用交叉验证，或者其他方式获取回归树的结果时，我们往往选择均方误差作为评估（在分类树中这个指标是score代表的预测准确率）。在随机森林回归器中追求的是MSE越小越好。
- **注意：**虽然均方误差永远为正，但是sklearn当中使用均方误差作为评判标准时，却是计算“负均方误差”（`neg_mean_squared_error`）。
- 这是因为sklearn在计算模型评估指标的时候，会考虑指标本身的性质，均方误差本身是一种误差，所以被sklearn划分为模型的一种损失(loss)，因此在sklearn当中，都以负数表示。真正的均方误差MSE的数值，其实就是`neg_mean_squared_error`去掉负号的数字。

回归类模型的评价指标

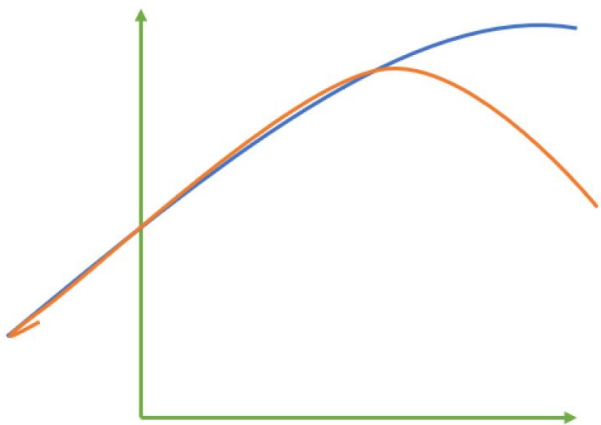
- 然而，回归树的接口`score`返回的是R平方，并不是MSE。
- 在我们的回归类算法中，我们有两种不同的角度来看待回归的效果：
- 第一，我们是否预测到了正确的数值。MSE、MAE。
- 第二，我们是否拟合到了足够的信息。R-Square。
- 这两种角度，分别对应着不同的模型评估指标。
- 注意和分类模型的区分：在分类型算法中，只有一种角度来评判，那就是是否预测到了正确的分类。

$$MSE = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

$$MAE = \frac{1}{m} \sum_{i=0}^{m-1} |y_i - \hat{y}_i|$$

回归类模型的评价指标

- 对于回归类算法而言，只探索数据预测是否准确是不够的。除了数据本身的数值大小之外，还希望模型能够捕捉到数据的“规律”，而是否捕获了这些信息并无法使用MSE、MAE来衡量。
- 蓝色线是真实标签，而橙色线是拟合模型。对于这样的一个拟合模型，如果使用MSE来对它进行判断，它的MSE会很小，因为大部分样本其实都被完美拟合了，少数样本的真实值和预测值的巨大差异在被均分到每个样本上之后，MSE就会很小。
- 但这样的拟合结果必然不是一个好结果，因为一旦新样本是处于拟合曲线的后半段的，预测结果必然会有巨大的偏差，而这不是我们希望看到的。所以，我们希望找到新的指标，除了判断预测的数值是否正确之外，还能够判断我们的模型是否拟合了足够多的，数值之外的信息。



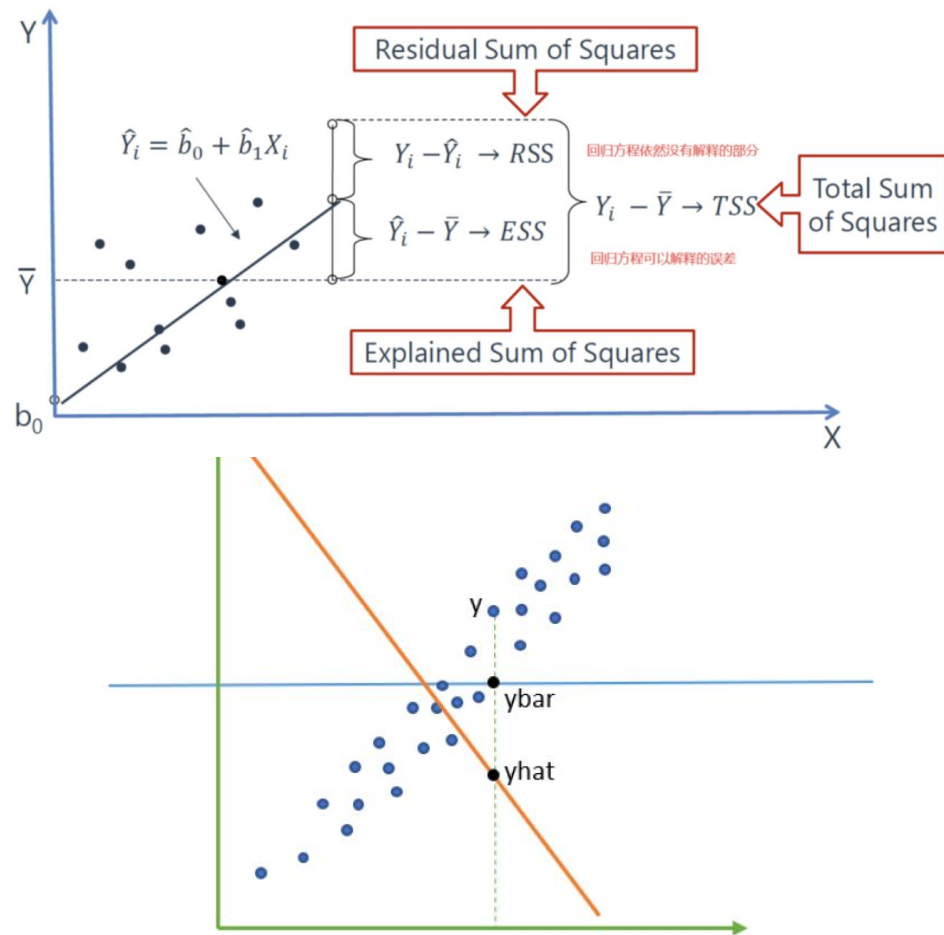
回归类模型的评价指标

- 方差=数据变异性=信息量，如果方差越大，代表数据上的信息量越多。
- 分子是真实值和预测值之差的差值，也就是我们的模型没有捕获到的信息总量。
- 分母是真实标签所带的信息量（Y的方差），所以其衡量的是1 - 模型没有捕获到的信息量占真实标签中所带的信息量的比例，所以，R-square越接近1越好。

$$R^2 = 1 - \frac{\sum_{i=0}^m (y_i - \hat{y}_i)^2}{\sum_{i=0}^m (y_i - \bar{y})^2} = 1 - \frac{RSS}{\sum_{i=0}^m (y_i - \bar{y})^2}$$

回归类模型的评价指标

$$\begin{aligned}TSS &= \sum_{i=0}^m (y_i - \bar{y})^2 \\&= \sum_{i=0}^m (y_i - \hat{y}_i + \hat{y}_i - \bar{y})^2 \\&= \sum_{i=0}^m (y_i - \hat{y}_i)^2 + \sum_{i=0}^m (\hat{y}_i - \bar{y})^2 + 2 \sum_{i=0}^m (y_i - \hat{y}_i)(\hat{y}_i - \bar{y}) \\&= RSS + ESS + 2 \sum_{i=0}^m (y_i - \hat{y}_i)(\hat{y}_i - \bar{y}) \\1 &= \frac{RSS}{TSS} + \frac{ESS}{TSS} + \frac{2 \sum_{i=0}^m (y_i - \hat{y}_i)(\hat{y}_i - \bar{y})}{TSS} \\1 - \frac{RSS}{TSS} &= \frac{ESS + 2 \sum_{i=0}^m (y_i - \hat{y}_i)(\hat{y}_i - \bar{y})}{TSS} \\R^2 &= \frac{ESS + 2 \sum_{i=0}^m (y_i - \hat{y}_i)(\hat{y}_i - \bar{y})}{TSS}\end{aligned}$$



- 蓝色的横线是均值线，橙色的线是模型，蓝色的点是样本点。现在对于来说，真实标签减预测值的值为正，但预测值却是一个负数，这说明，数据本身的均值，比对数据的拟合模型本身更接近数据的真实值，那模型是完全没有作用的，类似于分类模型中的分类准确率小于50%，不如瞎猜。

重要属性和接口

- 最重要的属性和接口，都与随机森林的分类器相一致，还是apply, fit, predict和score最为核心。值得一提的是，随机森林回归并没有predict_proba这个接口，因为对于回归来说，并不存在一个样本要被分到某个类别的概率问题，因此没有predict_proba这个接口。
- 随机森林回归器代码

```
from sklearn.datasets import load_boston
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestRegressor

boston = load_boston()
regressor = RandomForestRegressor(n_estimators=100, random_state=0)
cross_val_score(regressor, boston.data, boston.target, cv=10
                , scoring = "neg_mean_squared_error")

import sklearn
sklearn.metrics.get_scorer_names()
```


随机森林总结

- 随机森林是一种集成学习（Ensemble Learning）。Bagging的基础上（依然使用自助样本），再决策树的每个节点进行分裂时，仅随机选取部分变量（比如 m 个变量）作为候选的分裂变量。
- 它通过组合多个弱分类器，最终结果通过投票或取均值，使得整体模型的结果具有较高的精确度和泛化性能。其可以取得不错成绩，主要归功于“随机”和“森林”，一个使它具有抗过拟合能力，一个使它更加精准。
- 森林中的每棵树都是独立的，99.9%不相关的树做出的预测结果涵盖所有的情况，这些预测结果将会彼此抵消。
- 少数优秀的树的预测结果将会超脱于芸芸“噪音”，做出一个好的预测。
- 将若干个弱分类器的分类结果进行投票选择，从而组成一个强分类器，这就是随机森林Bagging的思想。

集成学习算法的基评估器不一定是决策树

- 可以使用决策树作为基本分类器。
- 也可以使用SVM、Logistic回归等其他分类器，习惯上，这些分类器组成的“总分类器”，仍然叫做随机森林。

随机森林优点

- 在当前的很多数据集上，相对其他算法有着很大的优势，表现良好。既能处理离散型数据，也能处理连续型数据，数据集无需规范化。
- 它能够处理很高维度（feature很多）的数据，并且不用做特征选择（降维）。
- 在训练完后，它能够给出哪些feature比较重要。在创建随机森林的时候，对generlization error使用的是无偏估计，模型泛化能力强。
- 训练速度快，容易做成并行化方法。训练时树与树之间是相互独立的。
- 如果有很大一部分的特征遗失，仍可以维持准确度。
- 双随机的引入，使得随机森林具有很好的抗噪声能力，并且使得随机森林不容易陷入过拟合。

随机森林缺点

- 随机森林已经被证明在某些噪音较大的分类或回归问题上会过拟合。
- 对于有不同取值的属性的数据，取值划分较多的属性会对随机森林产生更大的影响，所以随机森林在这种数据上产出的属性权值是不可信的。

随机森林在经济学中的应用

- Cicala S. Imperfect markets versus imperfect regulation in US electricity generation[J]. American Economic Review, 2022, 112(2): 409-441.
- 这项研究评估了美国将发电权从国家计划放开到市场决定所带来的收益。而逐年逐地区推行的政策，完全适用于标准的多时点双重差分的分析框架。
- 但是，这其中就出现外溢效应，而渠道可能是影响发电的重要因素——燃料价格。例如 A 地电力实行市场决定，相邻的 B 地电力实行国家计划。当 A 地电力可以实现市场交易时，可能会影响相邻地区的燃料价格，进而影响 B 地发电。而且因为产能原因，相同燃料价格的变动对不同地区的电量供给影响也是不一样的。因而，传统的双重差分法在不满足独立性条件假设、异质性对照组的情况下往往难以识别精准。研究者会合理地想到，如果能够通过自身历史数据构建自身的反事实，便能很好地解决上述问题。此时，便转化为预测问题，而预测是机器学习的优势所在，Cicala（2022）便利用随机森林预测了假如不是市场决定时的发电量的反事实结果。

机器学习因果推断之机器学习篇

从零开始精通机器学习应用

3.2随机森林（分类器、回归器）实战代码

I、缺失数据填补 II、调参的艺术

吕斌

上海外国语大学国际金融贸易学院

案例一：缺失数据填补

- 从现实中收集的数据几乎不可能是完美无缺的，往往都会有一些缺失值。面对缺失值，很多人选择的方式是直接删除含有缺失值的样本，这是一种有效的方法，但是有时候填补缺失值会比直接丢弃样本效果更好，即便我们其实并不知道缺失值的真实样貌。
- 在sklearn中，可以使用sklearn.impute.SimpleImputer来轻松地将均值，中值，或者其他最常用的数值填补到数据中，在案例中，将使用均值、0、随机森林回归来填补缺失值，并验证四种状况下的拟合状况，找出对使用的数据集来说最佳的缺失值填补方法。

案例一：缺失数据填补

- 1、导入库

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestRegressor

from sklearn.model_selection import cross_val_score
```

案例一：缺失数据填补

- 2、以加利福尼亚房价数据集为例，导入完整的数据集并探索

```
dataset = fetch_california_housing()

dataset.data.shape
#总共506*13=6578个数据

x_full, y_full = dataset.data, dataset.target
n_samples = x_full.shape[0]
n_features = x_full.shape[1]
```

案例一：缺失数据填补

- 3、为完整数据集放入缺失值

#首先确定我们希望放入的缺失数据的比例，在这里我们假设是50%，那总共就要有3289个数据缺失

```
rng = np.random.RandomState(0)
missing_rate = 0.5
n_missing_samples = int(np.floor(n_samples * n_features * missing_rate))
#np.floor向下取整，返回.0格式的浮点数
```

#所有数据要随机遍布在数据集的各行各列当中，而一个缺失的数据会需要一个行索引和一个列索引
#如果能够创建一个数组，包含3289个分布在0~506中间的行索引，和3289个分布在0~13之间的列索引，那我们就可以利用索引来为数据中的任意3289个位置赋空值
#然后我们用0，均值和随机森林来填写这些缺失值，然后查看回归的结果如何

```
missing_features = rng.randint(0,n_features,n_missing_samples)
missing_samples = rng.randint(0,n_samples,n_missing_samples)
```

```
#missing_samples = rng.choice(dataset.data.shape[0],n_missing_samples,replace=False)
```

#我们现在采样了3289个数据，远远超过我们的样本量506，所以我们使用随机抽取的函数randint。但如果我们需要的数据量小于我们的样本量506，那我们可以采用np.random.choice来抽样，choice会随机抽取不重复的随机数，因此可以帮助我们让数据更加分散，确保数据不会集中在一些行中

```
X_missing = X_full.copy()
y_missing = y_full.copy()
```

```
X_missing[missing_samples,missing_features] = np.nan
```

```
X_missing = pd.DataFrame(X_missing)
```

#转换成DataFrame是为了后续方便各种操作，numpy对矩阵的运算速度快到拯救人生，但是在索引等功能上却不如pandas来得好用

案例一：缺失数据填补

- 4、使用0和均值填补缺失值

#使用均值进行填补

```
from sklearn.impute import SimpleImputer  
imp_mean = SimpleImputer(missing_values=np.nan, strategy='mean')  
X_missing_mean = imp_mean.fit_transform(X_missing)
```

#使用0进行填补

```
imp_0 = SimpleImputer(missing_values=np.nan, strategy="constant", fill_value=0)  
X_missing_0 = imp_0.fit_transform(X_missing)
```


案例一：缺失数据填补

- 5、使用随机森林填补缺失值
- 任何回归都是从特征矩阵中学习，然后求解连续型标签 y 的过程，之所以能够实现这个过程，是因为回归算法认为，特征矩阵和标签之前存在着某种联系。实际上，标签和特征是可以相互转换的，比如说，在一个“用地区，环境，附近学校数量”预测“房价”的问题中，我们既可以用“地区”，“环境”，“附近学校数量”的数据来预测“房价”，也可以反过来，用“环境”，“附近学校数量”和“房价”来预测“地区”。而回归填补缺失值，正是利用了这种思想。

案例一：缺失数据填补

- 5、使用随机森林填补缺失值
- 对于一个有 n 个特征的数据来说，其中特征 T 有缺失值，就把特征 T 当作标签 Y ，其他的 $n-1$ 个特征和原本的标签组成新的特征矩阵。
- 特征 T 不缺失的值对应的其他 $n-1$ 个特征 + 本来的标签： X_{train}
- 特征 T 不缺失的值： Y_{train}
- 特征 T 缺失的值对应的其他 $n-1$ 个特征 + 本来的标签： X_{test}
- 特征 T 缺失的值： 未知，我们需要预测的 Y_{test}
- 这种做法，对于某一个特征大量缺失，其他特征却很完整的情况，非常适用。

案例一：缺失数据填补

- 那如果数据中除了特征T之外，其他特征也有缺失值怎么办？
- 答案是遍历所有的特征，从缺失最少的开始进行填补（因为填补缺失最少的特征所需要的准确信息最少）。
- 填补一个特征时，先将其他特征的缺失值用0代替，每完成一次回归预测，就将预测值放到原本的特征矩阵中，再继续填补下一个特征。每一次填补完毕，有缺失值的特征会减少一个，所以每次循环后，需要用0来填补的特征就越来越少。当进行到最后一个特征时（这个特征应该是所有特征中缺失值最多的），已经没有任何的其他特征需要用0来进行填补了，而我们已经使用回归为其他特征填补了大量有效信息，可以用来填补缺失最多的特征。遍历所有的特征后，数据就完整，不再有缺失值了。

案例一：缺失数据填补

- 5、使用随机森林填补缺失值

```
X_missing_reg = X_missing.copy()
sortindex = np.argsort(X_missing_reg.isnull().sum(axis=0)).values

for i in sortindex:

    #构建我们的新特征矩阵和新标签
    df = X_missing_reg
    fillc = df.iloc[:,i]
    df = pd.concat([df.iloc[:,df.columns != i],pd.DataFrame(y_full)],axis=1)

    #在新特征矩阵中，对含有缺失值的列，进行0的填补
    df_0 =SimpleImputer(missing_values=np.nan,
                        strategy='constant',fill_value=0).fit_transform(df)

    #找出我们的训练集和测试集
    Ytrain = fillc[fillc.notnull()]
    Ytest = fillc[fillc.isnull()]
    Xtrain = df_0[Ytrain.index,:]
    Xtest = df_0[Ytest.index,:]
```

案例一： 缺失数据填补

- 5、使用随机森林填补缺失值

```
#用随机森林回归来填补缺失值
```

```
rfc = RandomForestRegressor(n_estimators=100)
```

```
rfc = rfc.fit(Xtrain, Ytrain)
```

```
Ypredict = rfc.predict(Xtest)
```

```
#将填补好的特征返回到我们的原始的特征矩阵中
```

```
X_missing_reg.loc[X_missing_reg.iloc[:,i].isnull(),i] = Ypredict
```

案例一：缺失数据填补

- 6、对填补好的数据进行建模

#对所有数据进行建模，取得MSE结果

```
X = [X_full,X_missing_mean,X_missing_0,X_missing_reg]
```

```
mse = []
```

```
std = []
```

```
for x in X:
```

```
    estimator = RandomForestRegressor(random_state=0, n_estimators=100)
```

```
    scores = cross_val_score(estimator,x,y_full,scoring='neg_mean_squared_error',
```

```
cv=5).mean()
```

```
    mse.append(scores * -1)
```


案例一：缺失数据填补

- 7、用所得结果画出条形图

```
x_labels = ['Full data',
            'Zero Imputation',
            'Mean Imputation',
            'Regressor Imputation']
colors = ['r', 'g', 'b', 'orange']

plt.figure(figsize=(12, 6))
ax = plt.subplot(111)
for i in np.arange(len(mse)):
    ax.barh(i, mse[i], color=colors[i], alpha=0.6, align='center')
ax.set_title('Imputation Techniques with Boston Data')
ax.set_xlim(left=np.min(mse) * 0.9,
            right=np.max(mse) * 1.1)
ax.set_yticks(np.arange(len(mse)))
ax.set_xlabel('MSE')
ax.set_yticklabels(x_labels)
plt.show()
```

案例二：调参的艺术

- 为什么大多数机器学习教材讲解各种各样算法的原理和用途，但却对调参探究甚少？
- 这中间有许多原因：
 - 其一是因为，调参的方式总是根据数据的状况而定，所以没有办法一概而论；
 - 其二是因为，其实大家也都没有特别好的办法。

案例二：调参的艺术

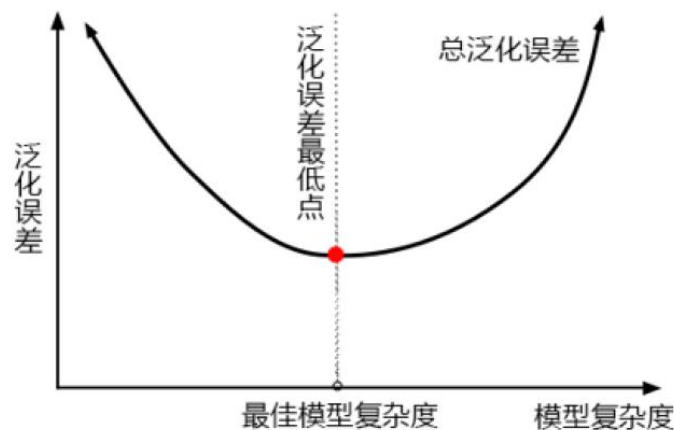
- 通过画学习曲线，或者网格搜索，我们能够探索到调参边缘（代价可能是训练一次模型要跑几周），但是在现实中，调参还是多依赖于经验。而这些经验，来源于：
 - (1) 非常正确的调参思路和方法
 - (2) 对模型评估指标的理解
 - (3) 对数据的感觉和经验
 - (4) 用洪荒之力去不断地尝试
- 在没有较多累积经验时，要学习对模型评估指标的理解和调参的思路。

案例二：调参的艺术

- 模型的正确调参思路，第一步是要找准目标：要做什么？
- 一般来说，这个目标是提升某个模型评估指标，比如对于随机森林来说，想要提升的是模型在未知数据上的准确率（由score或oob_score_来衡量）。
- 找准了这个目标就需要思考：模型在未知数据上的准确率受什么因素影响？在机器学习中，用来衡量模型在未知数据上的准确率的指标，叫做泛化误差（Generalization error）。

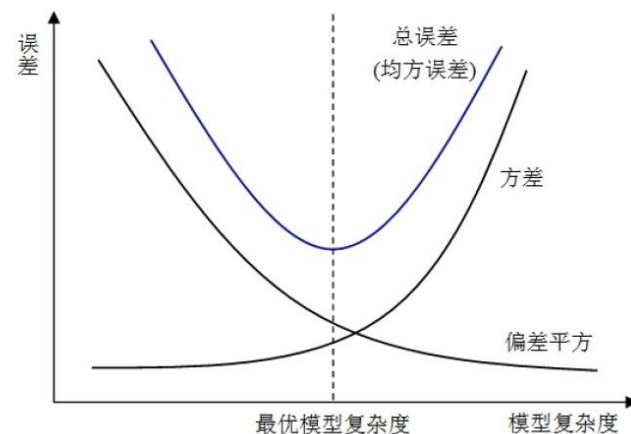
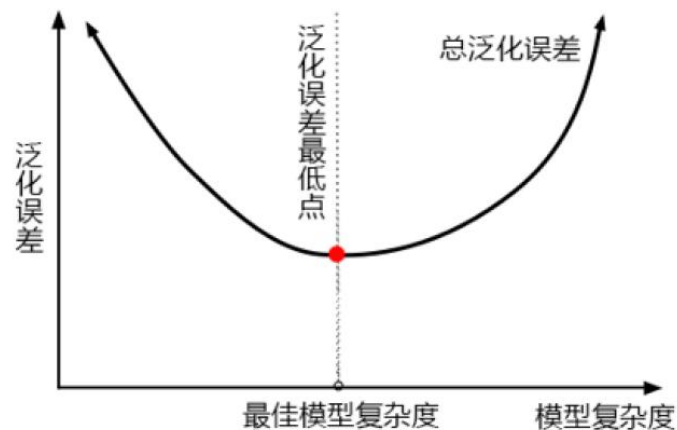
案例二：调参的艺术

- 泛化误差：
- 当模型在未知数据（测试集或者袋外数据）上表现糟糕时，模型的泛化程度不够，泛化误差大，模型的效果不好。泛化误差受到模型的结构（复杂度）影响。
- 下面这张图，它准确地描绘了泛化误差与模型复杂度的关系，当模型太复杂，模型就会过拟合，泛化能力就不够，所以泛化误差大。当模型太简单，模型就会欠拟合，拟合能力就不够，所以误差也会大。只有当模型的复杂度刚刚好，才能够达到泛化误差最小的目标。



案例二：调参的艺术

- 那模型的复杂度与参数有什么关系呢？
- 对树模型来说，树越茂盛，深度越深，枝叶越多，模型就越复杂。所以树模型是天生位于图的右上角的模型，随机森林是以树模型为基础，所以随机森林也是天生复杂度高的模型。随机森林的参数，都是向着一个目标去：减少模型的复杂度，把模型往图像的左边移动，防止过拟合。
- 当然了，调参没有绝对，也有天生处于图像左边的随机森林，所以调参之前，我们要先判断，模型现在究竟处于图像的哪一边。
- 泛化误差的背后其实是“偏差-方差困境”。

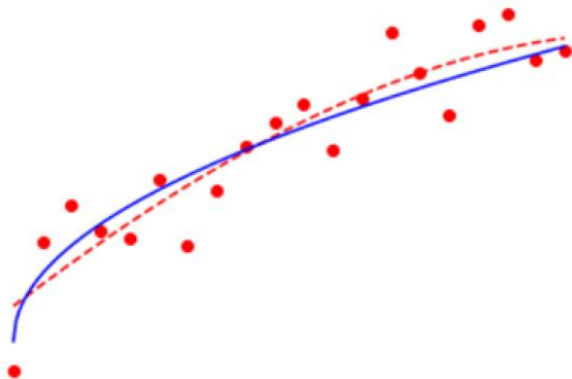


案例二：调参的艺术

- 一个集成模型(f)在未知数据集(D)上的泛化误差 $E(f; D)$ ，由方差(var)，偏差($bais$)和噪声(ϵ)共同决定。

$$E(f; D) = bias^2(x) + var(x) + \epsilon^2$$

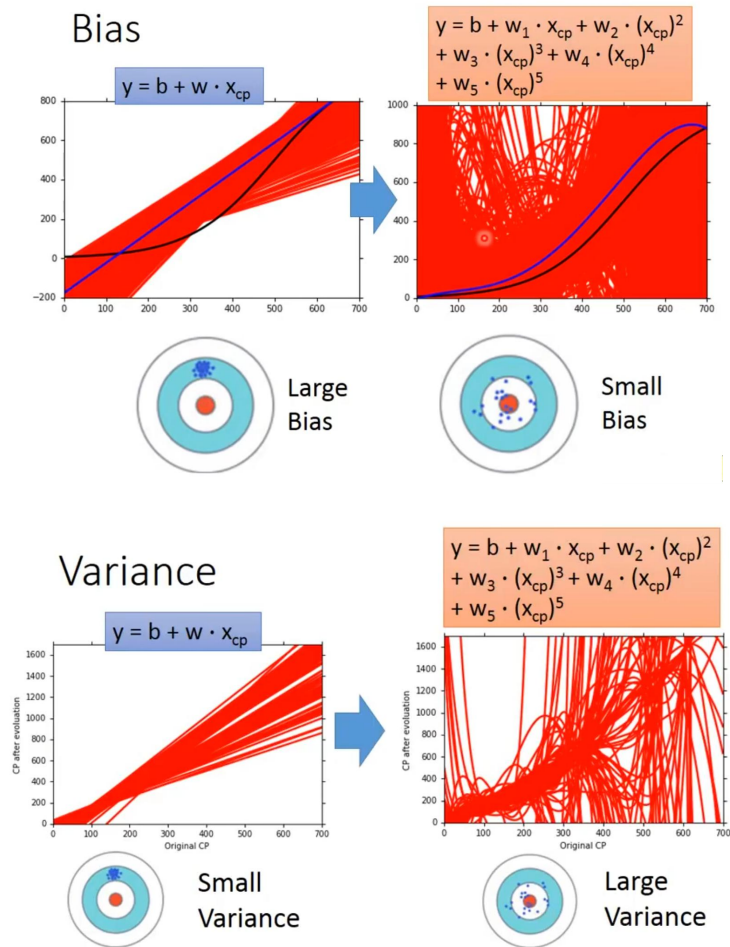
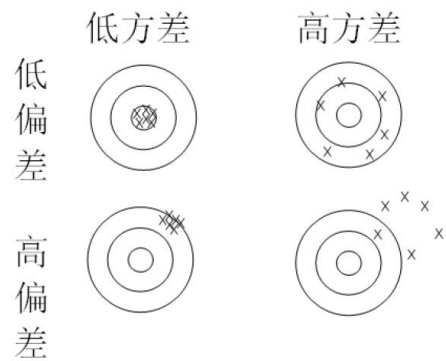
- 观察下面的图像，每个点就是集成算法中的一个基评估器产生的预测值。红色虚线代表着这些预测值的均值，而蓝色的线代表着数据本来的面貌。
- 偏差：模型的预测值与真实值之间的差异，即每一个红点到蓝线的距离。在集成算法中，每个基评估器都会有自己的偏差，集成评估器的偏差是所有基评估器偏差的均值。模型越精确，偏差越低。
- 方差：反映的是模型每一次输出结果与模型预测值的平均水平之间的误差，即每一个红点到红色虚线的距离，衡量模型的稳定性。模型越稳定，方差越低。



案例二：调参的艺术

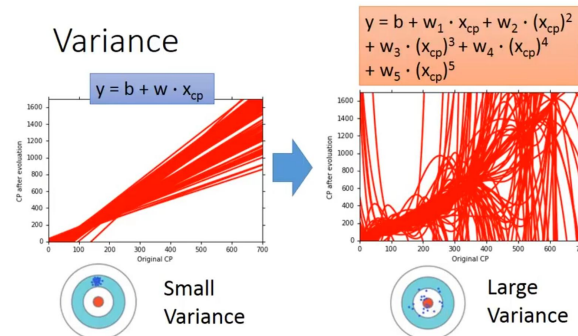
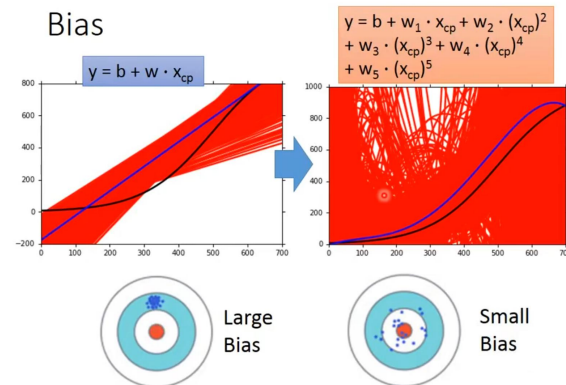
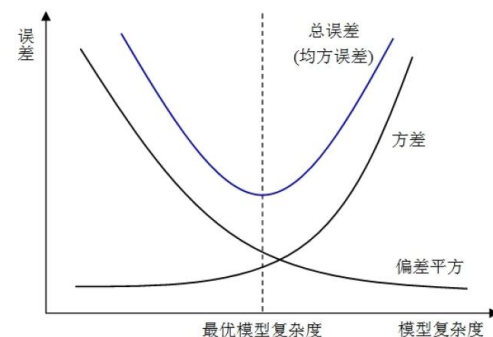
- 其中偏差衡量模型是否预测得准确，偏差越小，模型越“准”；而方差衡量模型每次预测的结果是否接近，即是说方差越小，模型越“稳”；噪声是机器学习无法干涉的部分。
- 一个好的模型，要对大多数未知数据都预测得“准”又“稳”。即是说，当偏差和方差都很低的时候，模型的泛化误差就小，在未知数据上的准确率就高。

	偏差大	偏差小
方差大	模型不适合这个数据 换模型	过拟合 模型很复杂 对某些数据集预测很准确 对某些数据集预测很糟糕
方差小	欠拟合 模型相对简单 预测很稳定 但对所有的数据预测都不太准确	泛化误差小，我们的目标



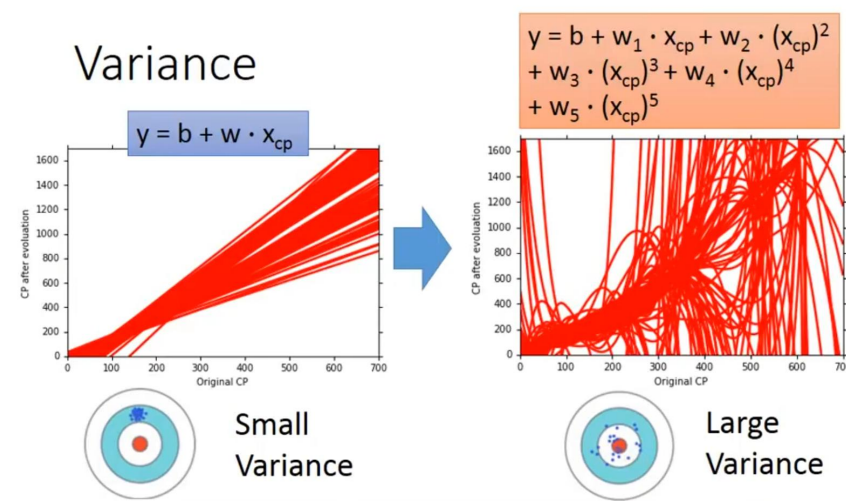
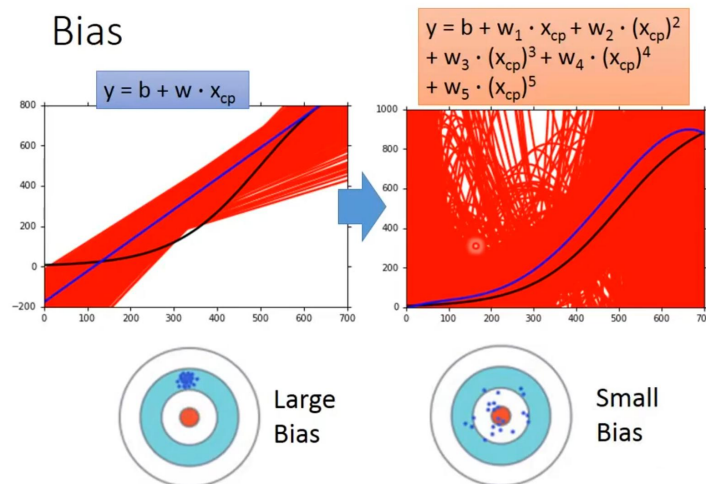
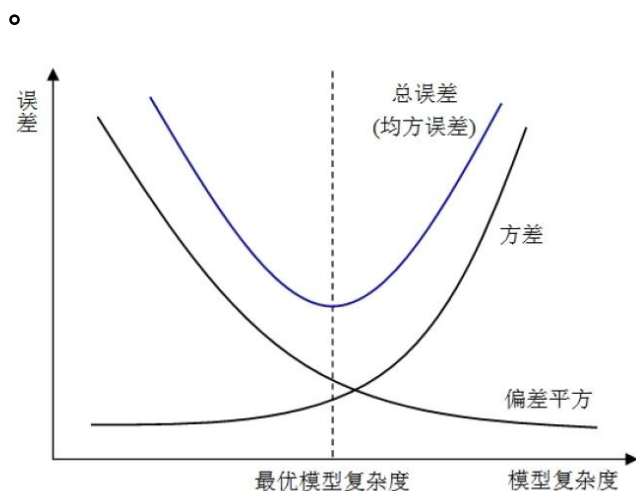
案例二：调参的艺术

- 从图上可以看出，模型复杂度大的时候，方差高，偏差低。偏差低，就是要求模型要预测得“准”。模型就会更努力去学习更多信息，会具体于训练数据，这会导致，模型在一部分数据上表现很好，在另一部分数据上表现却很糟糕。模型泛化性差，在不同数据上表现不稳定，所以方差就大。而尽量学习训练集，模型的建立必然更多细节，复杂程度必然上升。所以，复杂度高，方差高，总泛化误差高。
- 相对的，复杂度低的时候，方差低，偏差高。方差低，要求模型预测得“稳”，泛化性更强，那对于模型来说，它就不需要对数据进行一个太深的学习，只需要建立一个比较简单，判定比较宽泛的模型就可以了。结果就是，模型无法在某一类或者某一组数据上达成很高的准确度，所以偏差就会大。所以，复杂度低，偏差高，总泛化误差高。



案例二：调参的艺术

- 调参原则：
 - (1) 模型太复杂或者太简单，都会让泛化误差高，所以追求的是位于中间的平衡点。
 - (2) 模型太复杂就会过拟合，模型太简单就会欠拟合。
 - (3) 对树模型和树的集成模型来说，树的深度越深，枝叶越多，模型越复杂。
 - (4) 树模型和树的集成模型的目标，都是减少模型复杂度，把模型往图像的左边移动



案例二：调参的艺术

- 调参的目标是达到方差和偏差的完美平衡!
- 虽然方差和偏差不能同时达到最小值，但他们组成的泛化误差却可以有一个最低点，而就是要寻找这个最低点。对复杂度大的模型，要降低方差，对相对简单的模型，要降低偏差。
- 随机森林的基评估器都拥有较低的偏差和较高的方差，因为决策树本身是预测比较“准”，比较容易过拟合的模型，装袋法本身也要求基分类器的准确率必须要有50%以上。所以以随机森林为代表的装袋法的训练过程旨在降低方差，即降低模型复杂度，所以随机森林参数的默认设定都是假设模型本身在泛化误差最低点的右边。
- 所以，在降低复杂度的时候，本质其实是在降低随机森林的方差，随机森林所有的参数，也都是朝着降低方差的目标去。有了这一层理解，我们对复杂度和泛化误差的理解就更上一层楼了，对于我们调参，也有了更大的帮助。

案例二：调参的艺术

- 参数如何影响复杂度和模型？
- 一直以来调参都是在学习曲线上轮流找最优值，盼望能够将准确率修正到一个比较高的水平。然而现在了解到随机森林的调参方向：降低复杂度。那么就可以将那些对复杂度影响巨大的参数挑选出来，研究单调性，然后专注调整那些能最大限度让复杂度降低的参数。对于那些不单调的参数，或者反而会让复杂度升高的参数，就视情况使用，大多时候甚至可以退避。基于经验，对各个参数对模型的影响程度如右图排序。

参数	对模型在未知数据上的评估性能的影响	影响程度
n_estimators	提升至平稳，n_estimators↑，不影响单个模型的复杂度	☆☆☆☆
max_depth	有增有减，默认最大深度，即最高复杂度，向复杂度降低的方向调参max_depth↓，模型更简单，且向图像的左边移动	☆☆☆
min_samples_leaf	有增有减，默认最小限制1，即最高复杂度，向复杂度降低的方向调参min_samples_leaf↑，模型更简单，且向图像的左边移动	☆☆
min_samples_split	有增有减，默认最小限制2，即最高复杂度，向复杂度降低的方向调参min_samples_split↑，模型更简单，且向图像的左边移动	☆☆
max_features	有增有减，默认auto，是特征总数的开平方，位于中间复杂度，既可以向复杂度升高的方向，也可以向复杂度降低的方向调参max_features↓，模型更简单，图像左移 max_features↑，模型更复杂，图像右移 <i>max_features是唯一的，既能够让模型更简单，也能够让模型更复杂的参数，所以在调整这个参数的时候，需要考虑我们调参的方向</i>	☆
criterion	有增有减，一般使用gini	看具体情况

案例二：调参的艺术

- 案例：随机森林在乳腺癌数据上的调参
- 在学习完随机森林和机器学习中调参的基本思想后，了解了方差和偏差如何受到随机森林的参数们的影响。现在就使用基于方差和偏差的调参方法，在乳腺癌数据上进行一次随机森林的调参。
- 乳腺癌数据是sklearn自带的分类数据之一。案例中，往往使用真实数据，为什么我们要使用sklearn自带的数据库呢？因为真实数据在随机森林下的调参过程，往往非常缓慢。真实数据量大，维度高，在使用随机森林之前需要一系列的处理，因此不太适合用来做案例演示。
- 上节课案例中用的泰坦尼克号数据，用来调参的话需要很长时间，因此才选择sklearn当中自带的，结构相对清晰简单的数据来演示。

案例二：调参的艺术

- 1、导入需要的库

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

案例二：调参的艺术

- 2、导入数据集，探索数据

```
data = load_breast_cancer()
```

```
data
```

```
data.data.shape
```

```
data.target
```

#可以看到，乳腺癌数据集有569条记录，30个特征，单看维度虽然不算太高，但是样本量非常少。过拟合的情况可能存在。

案例二：调参的艺术

- 3、进行一次简单的建模，看看模型本身在数据集上的效果

```
rfc = RandomForestClassifier(n_estimators=100, random_state=90)
score_pre = cross_val_score(rfc, data.data, data.target, cv=10).mean()
```

```
score_pre
```

#这里可以看到，随机森林在乳腺癌数据上的表现本就还不错，在现实数据集上，基本上不可能什么都不调就看到95%以上的准确率

案例二：调参的艺术

- 4、随机森林调整的第一步：无论如何先来调n_estimators

```
"""
```

在这里我们选择学习曲线，可以使用网格搜索吗？可以，但是只有学习曲线，才能看见趋势

我个人的倾向是，要看见n_estimators在什么取值开始变得平稳，是否一直推动模型整体准确率的上升等信息

第一次的学习曲线，可以先用来帮助我们划定范围，我们取每十个数作为一个阶段，来观察n_estimators的变化如何引起模型整体准确率的变化

```
"""
```

```
##### 【TIME WARNING: 30 seconds】 #####
```

```
score1 = []
```

```
for i in range(0,200,10):
```

```
    rfc = RandomForestClassifier(n_estimators=i+1,
```

```
                                n_jobs=-1,
```

```
                                random_state=90)
```

```
    score = cross_val_score(rfc,data.data,data.target,cv=10).mean()
```

```
    score1.append(score)
```

```
print(max(score1),(score1.index(max(score1))*10)+1)
```

```
plt.figure(figsize=[20,5])
```

```
plt.plot(range(1,201,10),score1)
```

```
plt.show()
```

```
#list.index([object])
```

```
#返回这个object在列表list中的索引
```

案例二：调参的艺术

- 5、在确定好的范围内，进一步细化学习曲线
- 调整n_estimators的效果显著，模型的准确率立刻上升了0.005。接下来就进入网格搜索，我们将使用网格搜索对参数一个个进行调整。为什么我们不同时调整多个参数呢？原因有两个：
 - (1) 同时调整多个参数会运行非常缓慢，没有这么多的时间。
 - (2) 同时调整多个参数，会让我们无法理解参数的组合是怎么得来的，所以即便网格搜索调出来的结果不好，我们也不知道从哪里去改。在这里，为了使用复杂度-泛化误差方法（方差-偏差方法），对参数进行一个个地调整。

```
score1 = []
for i in range(35,45):
    rfc = RandomForestClassifier(n_estimators=i,
                                n_jobs=-1,
                                random_state=90)

    score = cross_val_score(rfc,data.data,data.target,cv=10).mean()
    score1.append(score)
print(max(score1),([*range(35,45)][score1.index(max(score1))]))
plt.figure(figsize=[20,5])
plt.plot(range(35,45),score1)
plt.show()
```


案例二：调参的艺术

"""

有一些参数是没有参照的，很难说清一个范围，这种情况下我们使用学习曲线，看趋势从曲线跑出的结果中选取一个更小的区间，再跑曲线

```
param_grid = {'n_estimators':np.arange(0, 200, 10)}
```

```
param_grid = {'max_depth':np.arange(1, 20, 1)}
```

```
param_grid = {'max_leaf_nodes':np.arange(25, 50, 1)}
```

对于大型数据集，可以尝试从1000来构建，先输入1000，每100个叶子一个区间，再逐渐缩小范围

有一些参数是可以找到一个范围的，或者说我们知道他们的取值和随着他们的取值，模型的整体准确率会如何变化，这样的参数我们就可以直接跑网格搜索

```
param_grid = {'criterion':['gini', 'entropy']}
```

```
param_grid = {'min_samples_split':np.arange(2, 2+20, 1)}
```

```
param_grid = {'min_samples_leaf':np.arange(1, 1+10, 1)}
```

```
param_grid = {'max_features':np.arange(5, 30, 1)}
```

"""

案例二：调参的艺术

- 7、开始按照参数对模型整体准确率的影响程度进行调参，首先调整max_depth

```
#调整max_depth

param_grid = {'max_depth':np.arange(1, 20, 1)}

# 一般根据数据的大小来进行一个试探，乳腺癌数据很小，所以可以采用1~10，或者1~20这样的试探
# 但对于像digit_recognition那样的大型数据来说，我们应该尝试30~50层深度（或许还不够）
# 更应该画出学习曲线，来观察深度对模型的影响

rfc = RandomForestClassifier(n_estimators=39
                             ,random_state=90
                             )
GS = GridSearchCV(rfc,param_grid,cv=10)
GS.fit(data.data,data.target)

GS.best_params_

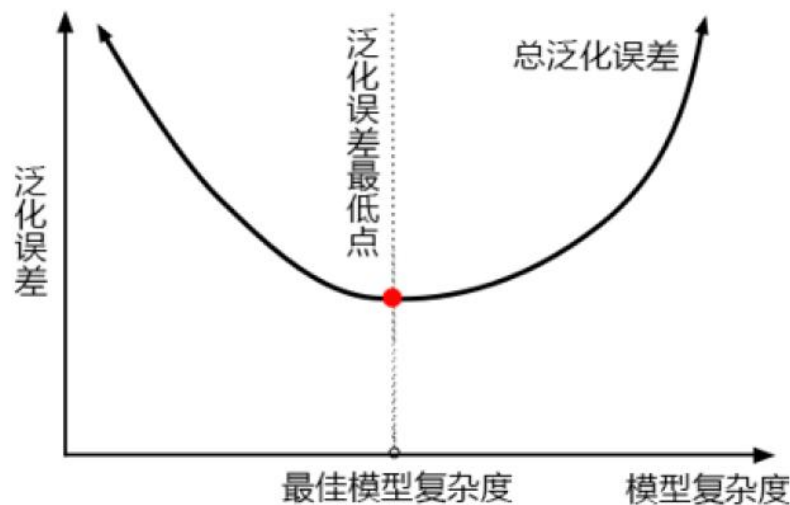
GS.best_score_
```

案例二：调参的艺术

- 在这里注意到，将`max_depth`设置为有限之后，模型的准确率下降了。限制`max_depth`，是让模型变得简单，把模型向左推，而模型整体的准确率下降了，即整体的泛化误差上升了，这说明模型现在位于图像左边，即泛化误差最低点的左边（偏差为主导的一边）。通常来说，随机森林应该在泛化误差最低点的右边，树模型应该倾向于过拟合，而不是拟合不足。这和数据集本身有关，但也有可能是我们调整的`n_estimators`对于数据集来说太大，
- 因此将模型拉到泛化误差最低点去了。然而，既然我们追求最低泛化误差，那我们就保留这个`n_estimators`，除非有其他因素，可以帮助我们达到更高的准确率。

案例二：调参的艺术

- 当模型位于图像左边时，我们需要的是增加模型复杂度（增加方差，减少偏差）的选项，因此 `max_depth` 应该尽量大，`min_samples_leaf` 和 `min_samples_split` 都应该尽量小。这几乎是在说明，除了 `max_features`，我们没有任何参数可以调整了，因为 `max_depth`，`min_samples_leaf` 和 `min_samples_split` 是剪枝参数，是减小复杂度的参数。
- 在这里，我们可以预言，我们已经非常接近模型的上限，模型很可能没有办法再进步了。那我们这就来调整一下 `max_features`，看看模型如何变化。



案例二：调参的艺术

- 8、调整max_features

```
#调整max_features
```

```
param_grid = {'max_features':np.arange(5,30,1)}
```

```
"""
```

max_features是唯一一个即能够将模型往左（低方差高偏差）推，也能够将模型往右（高方差低偏差）推的参数。我们需要根据调参前，模型所在的位置（在泛化误差最低点的左边还是右边）来决定我们要将max_features往哪边调。现在模型位于图像左侧，我们需要的是更高的复杂度，因此我们应该把max_features往更大的方向调整，可用的特征越多，模型才会越复杂。max_features的默认最小值是 $\sqrt{n_features}$ ，因此我们使用这个值作为调参范围的最小值。

```
"""
```

```
rfc = RandomForestClassifier(n_estimators=39  
                             , random_state=90  
                             )
```

```
GS = GridSearchCV(rfc,param_grid,cv=10)  
GS.fit(data.data,data.target)
```

```
GS.best_params_
```

```
GS.best_score_
```


案例二：调参的艺术

- 8、调整max_features
- 网格搜索返回了max_features的最小值，可见max_features升高之后，模型的准确率降低了。这说明，我们把模型往右推，模型的泛化误差增加了。前面用max_depth往左推，现在用max_features往右推，泛化误差都增加，这说明模型本身已经处于泛化误差最低点，已经达到了模型的预测上限，没有参数可以左右的部分了。剩下的那些误差，是噪声决定的，已经没有方差和偏差的舞台了。
- 如果是现实案例，到这一步其实就可以停下了，因为复杂度和泛化误差的关系已经告诉我们，模型不能再进步了。调参和训练模型都需要很长的时间，明知道模型不能进步了还继续调整，不是一个有效率的做法。如果我们希望模型更进一步，我们会选择更换算法，或者更换做数据预处理的方式。但是出于练习和探索的目的，继续调整我们的参数，让大家观察一下模型的变化，看看我们预测得是否正确。依然按照参数对模型整体准确率的影响程度进行调参。

案例二：调参的艺术

- 9、调整min_samples_leaf
- 可以看见，网格搜索返回了min_samples_leaf的最小值，并且模型整体的准确率还降低了，这和max_depth的情况一致，参数把模型向左推，但是模型的泛化误差上升了。在这种情况下，我们显然是不要把这个参数设置起来的，就让它默认就好了。

```
#调整min_samples_leaf

param_grid={'min_samples_leaf':np.arange(1, 1+10, 1)}

#对于min_samples_split和min_samples_leaf,一般是从他们的最小值开始向上增加10或20
#面对高维度高样本量数据, 如果不放心, 也可以直接+50, 对于大型数据, 可能需要200~300的范围
#如果调整的时候发现准确率无论如何都上不来, 那可以放心大胆调一个很大的数据, 大力限制模型的复杂度

rfc = RandomForestClassifier(n_estimators=39
                             ,random_state=90
                             )
GS = GridSearchCV(rfc,param_grid,cv=10)
GS.fit(data.data,data.target)

GS.best_params_

GS.best_score_
```

案例二：调参的艺术

- 10、继续尝试min_samples_split
- 和min_samples_leaf一样的结果，返回最小值并且模型整体的准确率降低了。

```
#调整min_samples_split

param_grid={'min_samples_split':np.arange(2, 2+20, 1)}

rfc = RandomForestClassifier(n_estimators=39
                             ,random_state=90
                             )
GS = GridsearchCV(rfc,param_grid,cv=10)
GS.fit(data.data,data.target)

GS.best_params_

GS.best_score_
```

案例二：调参的艺术

- 11、最后尝试一下criterion
- 和min_samples_leaf一样的结果，返回最小值并且模型整体的准确率降低了。

```
#调整criterion

param_grid = {'criterion':['gini', 'entropy']}

rfc = RandomForestClassifier(n_estimators=39
                             ,random_state=90
                             )
GS = GridSearchCV(rfc,param_grid,cv=10)
GS.fit(data.data,data.target)

GS.best_params_

GS.best_score_
```

案例二：调参的艺术

- 12、调整完毕，总结出模型的最佳参数

```
rfc = RandomForestClassifier(n_estimators=39, random_state=90)
score = cross_val_score(rfc, data.data, data.target, cv=10).mean()
score

score - score_pre
```

案例二：调参的艺术

- 总结:
- 在整个调参过程之中，首先调整了 `n_estimators`（无论如何都请先走这一步），然后调整 `max_depth`，通过 `max_depth` 产生的结果，来判断模型位于复杂度-泛化误差图像的哪一边，从而选择我们应该调整的参数和调参的方向。如果感到困惑，也可以画很多学习曲线来观察参数会如何影响我们的准确率，选取学习曲线中单调的部分来放大研究（如同我们对 `n_estimators` 做的）。学习曲线的拐点也许就是我们一直在追求的，最佳复杂度对应的泛化误差最低点（也是方差和偏差的平衡点）。
- 网格搜索也可以一起调整多个参数，大家只要有时间，可以自己跑一下，看看网格搜索会给我们怎样的结果，有时候，它的结果比我们的好，有时候，我们手动调整的结果会比较好。当然了，乳腺癌数据集非常完美，所以只需要调 `n_estimators` 一个参数就达到了随机森林在这个数据集上表现得极限。

案例二：调参的艺术

- 总结:
- 在上节课使用的泰坦尼克号案例数据中，使用同样的方法调出了如下的参数组合。
- 基于泰坦尼克号数据调整出来的参数，数据的处理过程请参考上一期的完整视频。这个组合的准确率达到了83.915%，比单棵决策树提升了大约7%，比调参前的随机森林提升了2.02%，这对于调参来说其实是一个非常巨大的进步。不过，泰坦尼克号数据的运行缓慢，大家量力而行，可以试试看用复杂度-泛化误差方法（方差-偏差方法）来解读一下这个调参结果和过程。

```
rfc = RandomForestClassifier(n_estimators=68
                             ,random_state=90
                             ,criterion="gini"
                             ,min_samples_split=8
                             ,min_samples_leaf=1
                             ,max_depth=12
                             ,max_features=2
                             ,max_leaf_nodes=36
                             )
```